ACTIVE-IA

Devolución de Trabajo Práctico

Materia:	PROG4A26 - Programacion 4 Agosto 2026
Comisión:	COMI-5 (2026)
Trabajo:	TP4 Programación IV — Módulo Proveedores (FastAPI CRUD)
Alumno:	Matias Manuel Carro

Calificación Final: 98/100

EVALUACIÓN POR CRITERIOS

C1: Estructura del módulo — 15/15

El módulo app/modules/proveedor/ está correctamente estructurado con routers.py, schemas.py y services.py, respetando la arquitectura modular del proyecto y delegando correctamente la lógica de negocio. Además, el router se encuentra adecuadamente registrado en main.py.

DESGLOSE POR SUBCRITERIO

C1.1 8/8 · OK
Se verificó la existencia del paquete app/modules/proveedor/ con __init__.py, routers.py, schemas.py y services.py. La lógica de negocio está completamente contenida en services.py.

C1.2 7/7 · OK
En app/main.py se importa y registra correctamente el router de proveedores mediante app.include_router(proveedor_router).

C2: Schemas Pydantic 13/15

Los esquemas Pydantic ProveedorBase, ProveedorCreate, ProveedorRead y ProveedorUpdate están bien definidos. Se observa un leve detalle en ProveedorBase donde email y telefono se definieron con default=None en lugar de default="".

DESGLOSE POR SUBCRITERIO

C2.1 6/8 · WARNING
Se definieron las validaciones para codigo, razon_social y cuit correctamente. Sin embargo, email y telefono utilizan default=None en lugar de default="" como especificaba la consigna.

C2.2 7/7 · OK
La jerarquía de herencia es adecuada: ProveedorCreate hereda de ProveedorBase, ProveedorRead añade el campo id y ProveedorUpdate define todos sus campos como opcionales.

C3: Endpoints CRUD 25/25

Todos los endpoints requeridos (POST, GET lista, GET detalle, PUT actualización total y PUT desactivación) están implementados con las rutas, prefijos, etiquetas y códigos de estado HTTP de éxito apropiados.

DESGLOSE POR SUBCRITERIO

C3.1 3/3 · OK
El APIRouter está configurado con prefix="/proveedores" y tags=["proveedores"].

C3.2 12/12 · OK
Endpoints POST /proveedores/ (201), GET /proveedores/ (200) y GET /proveedores/{id} (200) correctamente implementados.

C3.3 10/10 · OK
Endpoints PUT /proveedores/{id} (200) y PUT /proveedores/{id}/desactivar (200) implementados y delegando la lógica al servicio.

C4: Filtros y paginación 15/15

El endpoint de listado implementa paginación con skip y limit, así como filtrado por estado activo mediante query parameters correctamente validados.

DESGLOSE POR SUBCRITERIO

C4.1 7/7 · OK
Los parámetros skip (ge=0) y limit (ge=1, le=50) están definidos con sus valores por defecto y validaciones de rango requeridas.

C4.2 8/8 · OK
El parámetro activo (Optional[bool]) permite filtrar correctamente por proveedores activos, inactivos o todos, aplicando luego el slicing de paginación.

C5: Reglas de negocio y errores HTTP — 15/15

Se hacen cumplir todas las reglas de negocio (RN-01 a RN-05) devolviendo las excepciones HTTP y códigos correspondientes (422, 404 y 409).

DESGLOSE POR SUBCRITERIO

C5.1 6/6 · OK
RN-01 y RN-03 son validadas a nivel Pydantic respondiendo con status 422 Unprocessable Entity en caso de invalidez.

C5.2 3/3 · OK
RN-02 es verificada tanto en el alta como en la actualización levantando HTTPException 409 Conflict si el código ya existe.

C5.3 6/6 · OK
RN-04 lanza 404 Not Found si el id no existe y RN-05 lanza 409 Conflict si se intenta desactivar un proveedor que ya está inactivo.

C6: Archivos .http de prueba (REST Client) — 15/15

El archivo tests/test_api.http está completo e incluye pruebas para casos exitosos y de error (404, 409 y 422) cubriendo las reglas de negocio.

DESGLOSE POR SUBCRITERIO

C6.1 8/8 · OK
Existen peticiones HTTP para probar con éxito todos los endpoints del módulo de proveedores.

C6.2 7/7 · OK
Se incluyen pruebas explícitas que verifican los códigos de error HTTP 404, 409 y 422.

FORTALEZAS

- Excelente separación de responsabilidades entre capas (routers, schemas, services).
- Manejo robusto de reglas de negocio y errores HTTP específicos (404, 409, 422).
- Pruebas exhaustivas en test_api.http cubriendo tanto flujos principales como casos bordes.

RECOMENDACIONES

1. Asegurar que los valores por defecto en schemas coincidan exactamente con la especificación (por ejemplo, email y telefono con default="").

2. Utilizar el schema ProveedorUpdate en la firma del router si se prevé soportar actualizaciones parciales mediante PATCH.

COMENTARIOS DEL EVALUADOR

Excelente trabajo práctico. La arquitectura modular está muy bien implementada y respeta todos los patrones aprendidos en la materia. Las reglas de negocio y el manejo de excepciones HTTP están resueltos de forma sumamente prolija.